

Assignment Kit for Coding Standard



Personal Software Process for Engineers: Part I

The Software Engineering Institute (SEI)
is a federally funded research and development center
sponsored by the U.S. Department of Defense and
operated by Carnegie Mellon University.

This material is approved for public release.
Distribution limited by the Software Engineering Institute to attendees.

Personal Software Process for Engineers: Part I

Assignment Kit for the Coding Standard

Overview

Overview This assignment kit covers the following topics.

Section	See Page
Prerequisites	2
Objectives	2
Coding standard requirements	3
Example coding standard	4
Evaluation criteria and suggestions	7
Coding standard template	8

Prerequisites

Prerequisites

- Read Chapter 4
- Complete Size Counting Standard

Objectives

The objectives of the coding standard are to

- establish a consistent set of coding practices
- provide criteria for judging the quality of the code that you produce
- facilitate size counting by ensuring your programs are written so they can be readily counted
- for LOC counting, require that there be a separate physical line for each logical line of code

Coding standard requirements

Coding standard requirements

Produce, document, and submit a completed coding standard that calls for quality coding practices.

For LOC counting, ensure that a separate physical source line is used for each logical line of code.

Submit the coding standard with your program 2 assignment package.

Example coding Standard

Coding standard example

Pages 5 and 6 of this workbook contain an example C++ coding standard.

Notes about the example

- Since it is an example, tailor it to meet your personal needs.
 - If you have an existing organizational standard, consider using it for the PSP exercises.
-

Continued on next page

Example C++ Coding Standard

Purpose	To guide implementation of C++ programs
Program Headers	Begin all programs with a descriptive header.
Header Format	<pre> /* * Logic.java Version 3.0 14 Feb 2025 * * Copyright (c) 2025 Bridget Mendez. * All Rights Reserved. * * This software is the confidential and proprietary information of * Bridget Mendez ("Confidential Information"). You shall not disclose * such Confidential Information and shall use it only in accordance * with the terms of the license agreement you entered into with * Bridget Mendez. * * BRIDGET MENDEZ MAKES NO REPRESENTATIONS OR WARRANTIES * ABOUT THE * SUITABILITY OF THE SOFTWARE, EITHER EXPRESS OR IMPLIED, * INCLUDING * BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF * MERCHANTABILITY, * FITNESS FOR A PARTICULAR PURPOSE, OR NON-INFRINGEMENT. * BRIDGET * MENDEZ SHALL NOT BE LIABLE FOR ANY DAMAGES SUFFERED BY * LICENSEE * AS A RESULT OF USING, MODIFYING, OR DISTRIBUTING THIS * SOFTWARE OR * ITS DERIVATIVES. */ </pre>
Listing Contents	Provide a summary of the listing contents
Contents Example	<pre> //* * Listing Contents: * * Compilation instructions: * javac *.java * java App * * Classes in this file: * Logic * * Main methods: * Logic() * logic1a() // método que controla el menú y llama a SimpsonIntegration * * Related source files: * App.java * SimpsonIntegration.java * GammaFunction.java * Output.java * * /***** */ </pre>

(continued)

Example C++ Coding Standard (continued)

Reuse Instructions	- Describe how the program is used: declaration format, parameter values, types, and formats. - Provide warnings of illegal values, overflow conditions, or other conditions that could potentially result in improper operation.
Reuse Instruction Example	<pre> /** * Calcula la integral de la distribución t-Student desde 0 hasta x * usando la Regla de Simpson compuesta. * * Muestra en consola el valor de num_seg y p en cada iteración para * apoyar el análisis PSP y la depuración. * * @return valor aproximado de la integral p(x) para los parámetros * numSeg, e, dof y x configurados en el constructor. */ public double integrateWithDebug() { ... } </pre>
Identifiers	Use descriptive names for all variable, function names, constants, and other identifiers. Avoid abbreviations or single-letter variables.
Identifier Example	Clases: App, Logic, SimpsonIntegration, GammaFunction, Output. • Atributos: intNumSeg, dblE, intDOF, dblX, gammaValue, etc. • Métodos: logic1a, integrateWithDebug, ejecutarSimpson, computeCoefficient, computeDblGamma, writeData.
Comments	- Document the code so the reader can understand its operation. - Comments should explain both the purpose and behavior of the code. - Comment variable declarations to indicate their purpose.
Good Comment	// Valor inicial de la integral con el número de segmentos inicial. nuevo = ejecutarSimpson(this.intNumSeg);
Bad Comment	If(record_count > limit) /* check if record count exceeds limit */
Major Sections	Precede major program sections by a block comment that describes the processing done in the next section.
Example	<pre> /***** // ===== // NUEVA CONDICIÓN IMPORTANTE // ===== if (diferencia < 1E-17) { ... } </pre>
Blank Spaces	- Write programs with sufficient spacing so they do not appear crowded. - Separate every program construct with at least one space.
Indenting	- Indent each brace level from the preceding level. - Open and close braces should be on lines by themselves and aligned.
Indenting Example	<pre> public void logic1a() { Scanner scanner = new Scanner(System.in); System.out.println("=== MENU DE INTEGRACION ==="); ... if (opcion == 1) { System.out.println("\n=== INTEGRACION SIMPSON ==="); ... } } </pre>
Capitalization	- Capitalize all defines.

	- Lowercase all other identifiers and reserved words. - To make them readable, user messages may use mixed case.
Capitalization Examples	<pre>#define DEFAULT-NUMBER-OF-STUDENTS 15 int class-size = DEFAULT-NUMBER-OF-STUDENTS;</pre>

Evaluation criteria and suggestions

**Evaluation
criteria**

Your standard must be

- complete
 - legible
-

Suggestions

Keep your standards simple and short.

Do not hesitate to copy or build on the PSP materials.

Coding Standard Template

Purpose	To guide the development of programs
Program Headers	Begin all programs with a descriptive header.
Header Format	
Listing Contents	Provide a summary of the listing contents.
Contents Example	
Reuse Instructions	• Describe how the program is used. Provide the declaration format, parameter values and types, and parameter limits. • Provide warnings of illegal values, overflow conditions, or other conditions that could potentially result in improper operation.
Reuse Example	
Identifiers	Use descriptive names for all variables, function names, constants, and other identifiers. Avoid abbreviations or single letter variables.
Identifier Example	

(continued)

Coding Standard Template (continued)

Comments	• Document the code so that the reader can understand its operation. • Comments should explain both the purpose and behavior of the code. • Comment variable declarations to indicate their purpose.
Good Comment	
Bad Comment	
Major Sections	Precede major program sections by a block comment that describes the processing that is done in the next section
Example	
Blank Spaces	• Write programs with sufficient spacing so they do not appear crowded. • Separate every program construct with at least one space.
Indenting	• Indent every level of brace from the previous one. • Open and closing braces should be on lines by themselves and aligned with each other.
Indenting Example	
Capitalization	• Capitalized all defines. • Lowercase all other identifiers and reserved words. • Messages being output to the user can be mixed-case so as to make a clean user presentation.
Capitalization Example	